📚 EXPLICATION COMPLÈTE DU PROJET - Gestion de Projets et Tâches

🎯 C'EST QUOI CE PROJET ?

C'est une application web de **gestion de projets et de tâches** (comme Trello ou Asana).

Fonctionnalités principales :

- Créer un compte et se connecter
- Créer et gérer des projets
- Créer et assigner des tâches dans chaque projet
- Suivre l'avancement avec des statuts (À faire, En cours, Terminé)
- Ajouter des commentaires sur les tâches
- Voir un tableau de bord avec les statistiques

🏗️ ARCHITECTURE DU PROJET

L'application utilise **Angular 17** avec une architecture modulaire propre :

```
src/
├── app/
│   ├── core/          → Fonctionnalités centrales (services, guards, interceptors)
│   ├── features/      → Modules métier (auth, projects, tasks, dashboard)
│   ├── shared/        → Composants partagés (navbar, pipes, directives)
│   ├── app.routes.ts  → Configuration des routes
│   └── app.config.ts  → Configuration de l'application
```

📂 DÉTAIL DE CHAQUE DOSSIER

1️⃣ CORE (Le cœur de l'application)

Models (Modèles de données)

Les modèles définissent la structure des données dans l'application.

user.model.ts - Structure d'un utilisateur

```typescript
```

```
export interface User {
  id: number;          // Identifiant unique
  name: string;        // Nom complet
  email: string;       // Email
  role: 'admin' | 'member';  // Rôle (admin ou membre simple)
  createdAt: Date;     // Date de création du compte
}
```

project.model.ts - Structure d'un projet

```typescript
export interface Project {
  id: number;            // ID du projet
  name: string;          // Nom du projet
  description: string;   // Description
  status: 'active' | 'on-hold' | 'completed';  // Statut
  progress: number;      // Progression (0-100%)
  startDate?: Date;      // Date de début (optionnelle)
  endDate?: Date;        // Date de fin (optionnelle)
  createdAt: Date;       // Date de création
}
```

task.model.ts - Structure d'une tâche

```typescript
export interface Task {
  id: number;                          // ID unique
  title: string;                       // Titre
  description: string;                 // Description détaillée
  status: 'todo' | 'in-progress' | 'done';     // Statut
  priority: 'low' | 'medium' | 'high';         // Priorité
  projectId: number;                   // ID du projet parent
  assigneeId?: number;                 // ID de l'utilisateur assigné
  estimatedHours?: number;             // Heures estimées
  dueDate?: Date;                      // Date limite
  createdAt: Date;                     // Date de création
}
```

Services (Communication avec le backend)

Les services font les requêtes HTTP vers l'API backend.

auth.service.ts - Gestion de l'authentification

```typescript
// Ce service gère la connexion/déconnexion
export class AuthService {
  private apiUrl = 'http://localhost:3000/api';

  // Se connecter
  login(email: string, password: string): Observable<AuthResponse> {
    // Envoie email + mot de passe au serveur
    // Reçoit un token JWT en retour
    return this.http.post(`${this.apiUrl}/auth/login`, { email, password });
  }

  // S'inscrire
  register(name: string, email: string, password: string) {
    // Crée un nouveau compte utilisateur
    return this.http.post(`${this.apiUrl}/auth/register`, { name, email, password });
  }

  // Se déconnecter
  logout() {
    // Supprime le token stocké
    localStorage.removeItem('token');
  }

  // Vérifier si l'utilisateur est connecté
  isAuthenticated(): boolean {
    return !!localStorage.getItem('token');
  }
}
```

Comment ça marche ?

1. L'utilisateur entre email + mot de passe
2. Le service envoie ces infos au backend
3. Le backend vérifie et renvoie un **token JWT** (comme un badge d'accès)
4. Ce token est stocké dans `localStorage` du navigateur
5. Chaque requête suivante inclut ce token pour prouver l'identité

---

project.service.ts - Gestion des projets

```typescript
```

```typescript
export class ProjectService {
  private apiUrl = 'http://localhost:3000/api/projects';

  // Récupérer tous les projets
  getAll(): Observable<Project[]> {
    return this.http.get<Project[]>(this.apiUrl);
  }

  // Récupérer un projet par son ID
  getById(id: number): Observable<Project> {
    return this.http.get<Project>(`${this.apiUrl}/${id}`);
  }

  // Créer un nouveau projet
  create(project: Partial<Project>): Observable<Project> {
    return this.http.post<Project>(this.apiUrl, project);
  }

  // Modifier un projet existant
  update(id: number, project: Partial<Project>): Observable<Project> {
    return this.http.put<Project>(`${this.apiUrl}/${id}`, project);
  }

  // Supprimer un projet
  delete(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

Exemple d'utilisation :

```typescript
// Dans un composant
this.projectService.getAll().subscribe(projects => {
  console.log(projects); // Affiche tous les projets
});
```

---

task.service.ts - Gestion des tâches

```typescript
```

```typescript
export class TaskService {
  private apiUrl = 'http://localhost:3000/api/tasks';

  // Récupérer toutes les tâches d'un projet
  getByProject(projectId: number): Observable<Task[]> {
    return this.http.get<Task[]>(`${this.apiUrl}/project/${projectId}`);
  }

  // Créer une nouvelle tâche
  create(task: Partial<Task>): Observable<Task> {
    return this.http.post<Task>(this.apiUrl, task);
  }

  // Modifier une tâche
  update(id: number, task: Partial<Task>): Observable<Task> {
    return this.http.put<Task>(`${this.apiUrl}/${id}`, task);
  }

  // Supprimer une tâche
  delete(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

Guards (Protection des routes)

auth.guard.ts - Empêche l'accès aux pages si non connecté

```typescript
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // Vérifie si l'utilisateur est authentifié
  if (authService.isAuthenticated()) {
    return true;  // Laisse passer
  }

  // Sinon, redirige vers la page de login
  router.navigate(['/login']);
  return false;  // Bloque l'accès
};
```

Comment ça marche ?

- Quand tu essaies d'accéder à `/dashboard`, le guard vérifie d'abord si tu es connecté
- Si oui → tu peux accéder
- Si non → redirection automatique vers `/login`

Interceptors (Intercepent les requêtes HTTP)

auth.interceptor.ts - Ajoute le token à chaque requête

```typescript
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('token');

  if (token) {
    // Clone la requête et ajoute le header Authorization
    const clonedRequest = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
    return next(clonedRequest);
  }

  return next(req);
};
```

Pourquoi c'est utile ?

- Au lieu d'ajouter manuellement le token dans chaque requête
- L'interceptor le fait automatiquement pour TOUTES les requêtes
- Le backend peut ainsi vérifier l'identité de l'utilisateur

---

error.interceptor.ts - Gère les erreurs HTTP

```typescript
```

```typescript
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      let errorMessage = '';

      if (error.status === 401) {
        // Non autorisé - token expiré ou invalide
        errorMessage = 'Session expirée. Veuillez vous reconnecter.';
        // Rediriger vers login
      } else if (error.status === 404) {
        errorMessage = 'Ressource non trouvée.';
      } else if (error.status === 500) {
        errorMessage = 'Erreur serveur. Réessayez plus tard.';
      }

      console.error('Erreur HTTP:', errorMessage);
      return throwError(() => error);
    })
  );
};
```

Utilité :

- Centralise la gestion des erreurs
- Affiche des messages clairs selon le type d'erreur
- Peut rediriger automatiquement en cas de déconnexion

2️⃣ FEATURES (Modules métier)

A. Module AUTH (Authentification)

login.component.ts - Page de connexion

```typescript
```

```
export class LoginComponent {
  loginForm: FormGroup;  // Formulaire réactif Angular
  loading = false;       // État de chargement
  errorMessage = '';     // Message d'erreur

  constructor(
    private fb: FormBuilder,
    private authService: AuthService,
    private router: Router
  ) {
    // Création du formulaire avec validation
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  onSubmit(): void {
    if (this.loginForm.invalid) return;  // Ne rien faire si invalide

    this.loading = true;
    const { email, password } = this.loginForm.value;

    this.authService.login(email, password).subscribe({
      next: () => {
        this.router.navigate(['/dashboard']);  // Succès → tableau de bord
      },
      error: (error) => {
        this.errorMessage = 'Login échoué. Vérifiez vos identifiants.';
        this.loading = false;
      }
    });
  }
}
```

Le template HTML associé :

```
html
```

```
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  <!-- Champ Email -->
  <input
    type="email"
    formControlName="email"
    [class.error]="loginForm.get('email')?.invalid && loginForm.get('email')?.touched"
  />
  <!-- Message d'erreur si email invalide -->
  <div *ngIf="loginForm.get('email')?.invalid && loginForm.get('email')?.touched">
    Veuillez entrer un email valide
  </div>

  <!-- Champ Mot de passe -->
  <input
    type="password"
    formControlName="password"
  />

  <!-- Bouton Submit -->
  <button type="submit" [disabled]="loginForm.invalid || loading">
    {{ loading ? 'Connexion...' : 'Se connecter' }}
  </button>
</form>
```

Flux de connexion :

1. Utilisateur remplit email + mot de passe
2. Formulaire valide les données (email valide, mot de passe ≥ 6 caractères)
3. Si valide → appel à `authService.login()`
4. Backend vérifie les identifiants
5. Si correct → redirection vers `/dashboard`
6. Si incorrect → affiche un message d'erreur

---

register.component.ts - Page d'inscription

```typescript
```

```
export class RegisterComponent {
  registerForm: FormGroup;

  constructor(
    private fb: FormBuilder,
    private authService: AuthService,
    private router: Router
  ) {
    this.registerForm = this.fb.group({
      name: ['', [Validators.required, Validators.minLength(2)]],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  onSubmit(): void {
    if (this.registerForm.invalid) return;

    const { name, email, password } = this.registerForm.value;

    this.authService.register(name, email, password).subscribe({
      next: () => {
        this.router.navigate(['/dashboard']);
      },
      error: (error) => {
        this.errorMessage = 'Inscription échouée. Email déjà utilisé ?';
      }
    });
  }
}
```

Validations :

- Nom : minimum 2 caractères
- Email : format email valide
- Mot de passe : minimum 6 caractères

B. Module PROJECTS

project-list.component.ts - Liste des projets

```typescript
```

```
export class ProjectListComponent implements OnInit {
  projects: Project[] = [];
  showForm = false;

  constructor(
    private projectService: ProjectService
  ) {}

  ngOnInit(): void {
    this.loadProjects();  // Charger les projets au démarrage
  }

  loadProjects(): void {
    this.projectService.getAll().subscribe(projects => {
      this.projects = projects;
    });
  }

  openCreateForm(): void {
    this.showForm = true;  // Affiche le formulaire de création
  }

  onProjectCreated(projectData: any): void {
    this.projectService.create(projectData).subscribe({
      next: (newProject) => {
        this.projects.push(newProject);  // Ajoute à la liste
        this.showForm = false;           // Ferme le formulaire
      }
    });
  }

  deleteProject(id: number): void {
    if (confirm('Supprimer ce projet ?')) {
      this.projectService.delete(id).subscribe(() => {
        this.projects = this.projects.filter(p => p.id !== id);
      });
    }
  }
}
```

Le template :

```
html
```

```
<div class="projects-container">
  <button (click)="openCreateForm()">+ Nouveau Projet</button>

  <!-- Liste des projets -->
  <div class="project-card" *ngFor="let project of projects">
    <h3>{{ project.name }}</h3>
    <p>{{ project.description }}</p>
    <span class="status">{{ project.status }}</span>
    <button (click)="deleteProject(project.id)">Supprimer</button>
  </div>

  <!-- Formulaire modal -->
  <app-project-form
    *ngIf="showForm"
    (save)="onProjectCreated($event)"
    (cancel)="showForm = false"
  ></app-project-form>
</div>
```

---

project-form.component.ts - Formulaire de création/modification

```typescript
```

```
export class ProjectFormComponent implements OnInit {
  @Input() project: Project | null = null;  // Projet à modifier (null si création)
  @Output() save = new EventEmitter<any>();
  @Output() cancel = new EventEmitter<void>();

  projectForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.projectForm = this.fb.group({
      name: ['', Validators.required],
      description: ['', Validators.required],
      status: ['active'],
      progress: [0, [Validators.min(0), Validators.max(100)]],
      startDate: [''],
      endDate: ['']
    });
  }

  ngOnInit(): void {
    // Si on modifie un projet, pré-remplir le formulaire
    if (this.project) {
      this.projectForm.patchValue({
        name: this.project.name,
        description: this.project.description,
        status: this.project.status,
        progress: this.project.progress
      });
    }
  }

  onSubmit(): void {
    if (this.projectForm.invalid) return;

    const projectData = this.projectForm.value;
    this.save.emit(projectData);  // Envoie les données au parent
  }

  onCancel(): void {
    this.cancel.emit();  // Ferme le formulaire
  }
}
```

Communication parent-enfant :

- `@Input()` : reçoit des données du parent (le projet à modifier)
- `@Output()` : envoie des événements au parent (save, cancel)

C. Module TASKS

task-form.component.ts - Formulaire de tâche

```typescript
```

```
export class TaskFormComponent implements OnInit {
  @Input() task: Task | null = null;
  @Input() projectId!: number;  // ID du projet parent (obligatoire)
  @Output() save = new EventEmitter<any>();

  taskForm: FormGroup;
  users: User[] = [];  // Liste des utilisateurs pour l'assignation

  constructor(
    private fb: FormBuilder,
    private userService: UserService
  ) {
    this.taskForm = this.fb.group({
      title: ['', Validators.required],
      description: ['', Validators.required],
      status: ['todo'],
      priority: ['medium'],
      assigneeId: [null],          // Peut être null (non assigné)
      estimatedHours: [0],
      dueDate: ['']
    });
  }

  ngOnInit(): void {
    this.loadUsers();  // Charger la liste des utilisateurs

    if (this.task) {
      // Pré-remplir si modification
      this.taskForm.patchValue(this.task);
    }
  }

  loadUsers(): void {
    this.userService.getAll().subscribe(users => {
      this.users = users;
    });
  }

  onSubmit(): void {
    if (this.taskForm.invalid) return;

    const taskData = {
      ...this.taskForm.value,
      projectId: this.projectId  // Ajoute l'ID du projet
    };
```

```
    this.save.emit(taskData);
  }
}
```

Template avec sélection d'assigné :

```html
<form [formGroup]="taskForm" (ngSubmit)="onSubmit()">
  <input type="text" formControlName="title" placeholder="Titre" />
  <textarea formControlName="description" placeholder="Description"></textarea>

  <select formControlName="status">
    <option value="todo">À faire</option>
    <option value="in-progress">En cours</option>
    <option value="done">Terminé</option>
  </select>

  <select formControlName="priority">
    <option value="low">Basse</option>
    <option value="medium">Moyenne</option>
    <option value="high">Haute</option>
  </select>

  <!-- Liste déroulante des utilisateurs -->
  <select formControlName="assigneeId">
    <option [value]="null">Non assigné</option>
    <option *ngFor="let user of users" [value]="user.id">
      {{ user.name }}
    </option>
  </select>

  <button type="submit">Enregistrer</button>
</form>
```

D. Module DASHBOARD

dashboard-page.component.ts - Tableau de bord

```typescript
```

```
export class DashboardPageComponent implements OnInit {
  projects: Project[] = [];
  tasks: Task[] = [];
  stats = {
    totalProjects: 0,
    activeProjects: 0,
    totalTasks: 0,
    completedTasks: 0
  };

  constructor(
    private projectService: ProjectService,
    private taskService: TaskService
  ) {}

  ngOnInit(): void {
    this.loadDashboardData();
  }

  loadDashboardData(): void {
    // Charger tous les projets
    this.projectService.getAll().subscribe(projects => {
      this.projects = projects;
      this.calculateStats();
    });
  }

  calculateStats(): void {
    this.stats.totalProjects = this.projects.length;
    this.stats.activeProjects = this.projects.filter(p => p.status === 'active').length;

    // Calculer les tâches de tous les projets
    let allTasks: Task[] = [];
    this.projects.forEach(project => {
      this.taskService.getByProject(project.id).subscribe(tasks => {
        allTasks = [...allTasks, ...tasks];
        this.stats.totalTasks = allTasks.length;
        this.stats.completedTasks = allTasks.filter(t => t.status === 'done').length;
      });
    });
  }
}
```

Template du dashboard :

```
html
```

```
<div class="dashboard">
  <h1>Tableau de bord</h1>

  <!-- Cartes de statistiques -->
  <div class="stats-grid">
    <div class="stat-card">
      <h3>{{ stats.totalProjects }}</h3>
      <p>Projets totaux</p>
    </div>

    <div class="stat-card">
      <h3>{{ stats.activeProjects }}</h3>
      <p>Projets actifs</p>
    </div>

    <div class="stat-card">
      <h3>{{ stats.totalTasks }}</h3>
      <p>Tâches totales</p>
    </div>

    <div class="stat-card">
      <h3>{{ stats.completedTasks }}</h3>
      <p>Tâches complétées</p>
    </div>
  </div>

  <!-- Liste des projets récents -->
  <div class="recent-projects">
    <h2>Projets récents</h2>
    <div *ngFor="let project of projects.slice(0, 5)">
      <a [routerLink]="['/projects', project.id]">{{ project.name }}</a>
    </div>
  </div>
</div>
```

3️⃣ SHARED (Composants partagés)

navbar.component.ts - Barre de navigation

```typescript
```

```
export class NavbarComponent {
  currentUser: User | null = null;

  constructor(
    private authService: AuthService,
    private router: Router
  ) {
    // Récupérer l'utilisateur connecté
    this.authService.getCurrentUser().subscribe(user => {
      this.currentUser = user;
    });
  }

  logout(): void {
    this.authService.logout();
    this.router.navigate(['/login']);
  }
}
```

Template :

```html
<nav class="navbar">
  <div class="logo">Project Manager</div>

  <ul class="nav-links">
    <li><a routerLink="/dashboard">Tableau de bord</a></li>
    <li><a routerLink="/projects">Projets</a></li>
  </ul>

  <div class="user-menu" *ngIf="currentUser">
    <span>{{ currentUser.name }}</span>
    <button (click)="logout()">Déconnexion</button>
  </div>
</nav>
```

Pipes (Transforment l'affichage des données)

time-ago.pipe.ts - Affiche "il y a X temps"

```typescript
```

```typescript
@Pipe({ name: 'timeAgo' })
export class TimeAgoPipe implements PipeTransform {
  transform(value: Date): string {
    const now = new Date();
    const diff = now.getTime() - new Date(value).getTime();

    const minutes = Math.floor(diff / 60000);
    const hours = Math.floor(minutes / 60);
    const days = Math.floor(hours / 24);

    if (days > 0) return `il y a ${days} jour${days > 1 ? 's' : ''}`;
    if (hours > 0) return `il y a ${hours} heure${hours > 1 ? 's' : ''}`;
    if (minutes > 0) return `il y a ${minutes} minute${minutes > 1 ? 's' : ''}`;
    return 'à l\'instant';
  }
}
```

Utilisation :

```html
<p>Créé {{ task.createdAt | timeAgo }}</p>
<!-- Affichera : "Créé il y a 2 heures" -->
```

---

filter-status.pipe.ts - Filtre les tâches par statut

```typescript
@Pipe({ name: 'filterStatus' })
export class FilterStatusPipe implements PipeTransform {
  transform(tasks: Task[], status: string): Task[] {
    if (!status) return tasks;
    return tasks.filter(task => task.status === status);
  }
}
```

Utilisation :

```html
<div *ngFor="let task of tasks | filterStatus:'done'">
  {{ task.title }}
</div>
<!-- N'affiche que les tâches terminées -->
```

Directives

highlight.directive.ts - Surligne au survol

```typescript
@Directive({ selector: '[appHighlight]' })
export class HighlightDirective {
  constructor(private el: ElementRef) {}

  @HostListener('mouseenter') onMouseEnter() {
    this.el.nativeElement.style.backgroundColor = '#f0f0f0';
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.el.nativeElement.style.backgroundColor = '';
  }
}
```

Utilisation :

```html
<div appHighlight>Passe ta souris ici</div>
<!-- Le fond devient gris au survol -->
```

🔐 SYSTÈME D'AUTHENTIFICATION - DÉTAIL COMPLET

Comment fonctionne le JWT (JSON Web Token) ?

1. **Connexion :**

```
Utilisateur → email + password → Backend
Backend vérifie → Si OK, génère un JWT → Renvoie le token
Frontend stocke le token dans localStorage
```

2. **Structure d'un JWT :**

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjEsImVtYWlsIjoidGVzdEB0ZXN0LmNvbSJ9.abc123xyz
│                                    │                                    │
Header (algorithme)                  Payload (données user)               Signature
```

3. **Utilisation du token :**

```
Chaque requête HTTP → Header: Authorization: Bearer <token>
Backend décode le token → Vérifie la signature → Identifie l'utilisateur
```

4. **Sécurité :**

- Le token est signé cryptographiquement
- Impossible de le modifier sans invalider la signature
- Expire après un certain temps (ex: 24h)

🔄 FLUX DE DONNÉES COMPLET

Exemple : Créer une nouvelle tâche

```
1. Utilisateur clique sur "Nouvelle tâche"
   ↓
2. Component affiche le formulaire (task-form.component)
   ↓
3. Utilisateur remplit : titre, description, priorité
   ↓
4. Utilisateur clique "Enregistrer"
   ↓
5. Component valide le formulaire (Validators)
   ↓
6. Component émet l'événement save avec les données
   ↓
7. Component parent reçoit l'événement
   ↓
8. Appel à taskService.create(taskData)
   ↓
9. Service fait une requête HTTP POST vers l'API
   Headers: { Authorization: Bearer <token> }  ← Ajouté par l'interceptor
   ↓
10. Backend reçoit la requête
    - Vérifie le token
    - Enregistre la tâche en base de données
    - Renvoie la tâche créée avec son ID
   ↓
11. Service reçoit la réponse
   ↓
12. Component reçoit la nouvelle tâche via subscribe()
```

↓
13. Component ajoute la tâche à la liste affichée
↓
14. Component ferme le formulaire
↓
15. L'utilisateur voit la nouvelle tâche dans la liste

🎨 VALIDATION DES FORMULAIRES

Types de validateurs Angular

```typescript
// Validateurs intégrés
Validators.required            // Champ obligatoire
Validators.email               // Format email valide
Validators.minLength(6)        // Minimum 6 caractères
Validators.maxLength(100)      // Maximum 100 caractères
Validators.min(0)              // Valeur minimum 0
Validators.max(100)            // Valeur maximum 100
Validators.pattern(/regex/)    // Correspond à un pattern

// Exemple d'utilisation
this.form = this.fb.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(6)]],
  age: ['', [Validators.required, Validators.min(18), Validators.max(99)]]
});
```

Affichage des erreurs

```typescript
// Dans le component
get email() {
  return this.form.get('email');
}

hasError(fieldName: string, errorType: string): boolean {
  const field = this.form.get(fieldName);
  return field?.hasError(errorType) && field?.touched;
}
```

```html
```

```
<!-- Dans le template -->
<input type="email" formControlName="email" />

<!-- Afficher les erreurs spécifiques -->
<div *ngIf="hasError('email', 'required')">
  L'email est obligatoire
</div>
<div *ngIf="hasError('email', 'email')">
  L'email n'est pas valide
</div>

<!-- Ou vérification directe -->
<div *ngIf="email?.invalid && email?.touched">
  <span *ngIf="email?.errors?.['required']">Email requis</span>
  <span *ngIf="email?.errors?.['email']">Format invalide</span>
</div>
```

📡 OBSERVABLES & RXJS

C'est quoi un Observable ?

Un Observable est comme un **flux de données** qui peut émettre des valeurs au fil du temps.

```typescript
// Exemple simple
const observable = new Observable(observer => {
  observer.next('Valeur 1');
  observer.next('Valeur 2');
  observer.complete();
});

// S'abonner pour recevoir les valeurs
observable.subscribe({
  next: (value) => console.log(value),
  error: (err) => console.error(err),
  complete: () => console.log('Terminé')
});

// Affiche :
// Valeur 1
// Valeur 2
// Terminé
```

Opérateurs RxJS courants

```typescript
import { map, filter, debounceTime, switchMap } from 'rxjs/operators';

// map - Transformer les valeurs
this.http.get<User[]>('/api/users').pipe(
  map(users => users.map(u => u.name))  // Garde uniquement les noms
);

// filter - Filtrer les valeurs
clicks$.pipe(
  filter(click => click.x > 100)  // Garde uniquement si x > 100
);

// debounceTime - Attendre X ms avant d'émettre
searchInput$.pipe(
  debounceTime(300)  // Attend 300ms après la dernière frappe
);

// switchMap - Annule la requête précédente
searchInput$.pipe(
  debounceTime(300),
  switchMap(term => this.http.get(`/api/search?q=${term}`))
);
```

Exemple pratique : Recherche en temps réel

```typescript
export class SearchComponent implements OnInit {
  searchControl = new FormControl('');
  results$: Observable<any[]>;

  ngOnInit() {
    this.results$ = this.searchControl.valueChanges.pipe(
      debounceTime(300),      // Attend 300ms après frappe
      distinctUntilChanged(), // Ignore si valeur identique
      switchMap(term =>       // Fait la recherche
        this.http.get<any[]>(`/api/search?q=${term}`)
      )
    );
  }
}
```

```html
<input [formControl]="searchControl" placeholder="Rechercher..." />
<div *ngFor="let result of results$ | async">
  {{ result.name }}
</div>
```

🗺️ ROUTING (Navigation)

app.routes.ts

```typescript
export const routes: Routes = [
  { path: '', redirectTo: '/login', pathMatch: 'full' },
  { path: 'login', component: LoginComponent },
  { path: 'register', component: RegisterComponent },
  {
    path: 'dashboard',
    component: DashboardPageComponent,
    canActivate: [authGuard]  // Protégé par le guard
  },
  {
    path: 'projects',
    component: ProjectListComponent,
    canActivate: [authGuard]
  },
  {
    path: 'projects/:id',
    component: ProjectDetailComponent,
    canActivate: [authGuard]
  },
  { path: '**', redirectTo: '/login' }  // Route par défaut si URL inconnue
];
```

Navigation programmatique

```typescript
```

```
// Dans un component
constructor(private router: Router) {}

goToProject(id: number) {
  this.router.navigate(['/projects', id]);
  // Navigue vers /projects/123
}

goBack() {
  this.router.navigate(['..']);
  // Remonte d'un niveau
}
```

Récupérer les paramètres d'URL

```typescript
// URL: /projects/123
constructor(private route: ActivatedRoute) {}

ngOnInit() {
  const projectId = this.route.snapshot.params['id'];
  console.log(projectId);  // "123"

  // Ou de manière réactive
  this.route.params.subscribe(params => {
    const id = params['id'];
  });
}
```

🎯 CONCEPTS CLÉS À RETENIR POUR TA VALIDATION

1. Architecture en couches

```
Présentation (Components) → Logique métier (Services) → Données (API/Backend)
```

2. Injection de dépendances

```typescript
```

```
// Angular injecte automatiquement les services dans le constructeur
constructor(
  private authService: AuthService,  // Pas besoin de new AuthService()
  private router: Router
) {}
```

3. Communication parent-enfant

```typescript
// Enfant → Parent : @Output() + EventEmitter
@Output() save = new EventEmitter<any>();
this.save.emit(data);

// Parent → Enfant : @Input()
@Input() project: Project;
```

4. Formulaires réactifs

```typescript
// Création
this.form = this.fb.group({
  email: ['', Validators.required]
});

// Validation
if (this.form.invalid) return;

// Récupération des valeurs
const data = this.form.value;
```

5. Cycle de vie des components

```typescript
ngOnInit()       // Au démarrage (idéal pour charger les données)
ngOnChanges()    // Quand un @Input() change
ngOnDestroy()    // Avant la destruction (nettoyage)
```

6. Observables

```typescript
```

```
// Créer
this.http.get('/api/data')

// S'abonner
.subscribe({
  next: (data) => { /* Succès */ },
  error: (err) => { /* Erreur */ },
  complete: () => { /* Terminé */ }
})
```

📝 QUESTIONS TYPIQUES D'UN PROF

Q: Pourquoi utiliser des services ?

R: Pour centraliser la logique métier, réutiliser le code, et séparer les responsabilités.

Q: C'est quoi un guard ?

R: Un guard protège les routes. Il vérifie des conditions (ex: utilisateur connecté) avant d'autoriser l'accès.

Q: Différence entre FormGroup et FormControl ?

R: FormControl = un seul champ. FormGroup = groupe de FormControls (un formulaire complet).

Q: Pourquoi utiliser HttpInterceptor ?

R: Pour intercepter toutes les requêtes HTTP et ajouter automatiquement des headers (comme le token d'authentification).

Q: C'est quoi le lifecycle hook ngOnInit ?

R: Une méthode appelée automatiquement quand le component est initialisé. Idéal pour charger les données.

Q: Différence entre Observable et Promise ?

R: Observable peut émettre plusieurs valeurs, Promise une seule. Observable est annulable, Promise non.

🚀 RÉSUMÉ ULTRA-SIMPLE

1. **Angular** = Framework pour créer des applications web
2. **Components** = Blocs visuels réutilisables (comme des Lego)
3. **Services** = Logique métier partagée entre components
4. **Guards** = Protègent les routes
5. **Interceptors** = Modifient automatiquement les requêtes HTTP
6. **Observables** = Flux de données asynchrones

7. **Routing** = Navigation entre pages
8. **Forms** = Gestion des formulaires avec validation
9. **JWT** = Token d'authentification sécurisé

Flux global :

```
User → Component → Service → HTTP → Backend → Database
                ↓
            Interceptor ajoute le token
                ↓
            Guard vérifie l'accès
```

Bonne chance pour ta validation ! 🎓